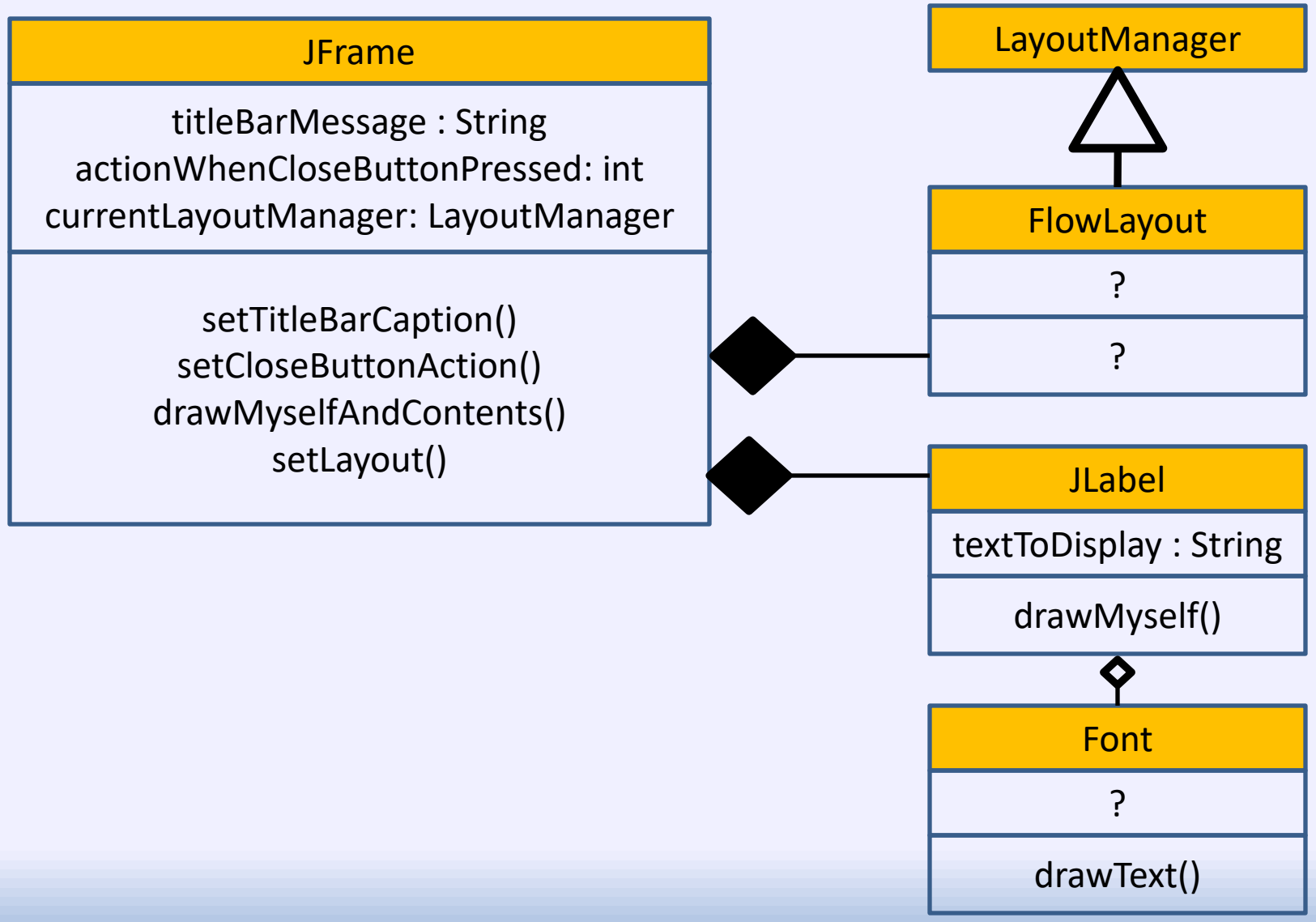
COMP1009
Programming Paradigms

Lecture OO-8
Layout managers and Labels

Last Lecture

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class GUIHelloWorld  { // Just the one class
    public static void main(String[] args)  { // Same main function as before – static
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Close window exits
        guiFrame.setTitle("Hello World!"); // Set a caption/title bar content for the frame
        guiFrame.setLocationRelativeTo(null);  // centre on screen
        guiFrame.getContentPane().setLayout( new FlowLayout() );
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!"); // Set its message to show
        label.setFont(new Font("Ariel", Font.BOLD, 30)); // Set a font for the label
        guiFrame.getContentPane().add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

Class diagram from before



Overview/summary of OO

1. Identify the classes you need
   - Understand your class library if you are using one
2. Create the objects – using existing or new classes
3. Customise the objects
   - Creating any missing objects that you need for this
4. Call operations on the objects to get them to do what you need them to do

- In OO programming you are usually:
  - Using existing classes (so many Java classes online!)
  - Creating or adapting classes (when flexibility insufficient)
    - Aggregation or inheritance
  - Creating and configuring objects of these classes
  - Plugging the objects together
- Reduces code, reduces debugging, shares the workload

Comments: threads and main()

- When a window is made visible by Swing, a thread will be started, but the main() thread will also still be running
- I don't want to go through threads with you, because it will confuse some people
  - If a program has X threads, it can be doing X things at once
  - Problems occur when threads try to change the same things at the same time
  - There are standard Java solutions for this though
- **To avoid any problems:**
  - **Let main() end once you show a window, so that you don't try to do two things with the same resource at once**
  - **Don't try to alter (e.g. call methods on) the GUI objects using main thread once a window is visible**
  - **Note: Swing is NOT "thread-safe"!**

This Lecture

- JLabel
- ColorLabel
- LayoutManagers and JPanel

Reminder: The code which uses a label

```
JFrame guiFrame = new JFrame();
guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
guiFrame.setTitle("Hello World!");
guiFrame.getContentPane().setLayout(new FlowLayout());
guiFrame.getContentPane().add(create());
guiFrame.getContentPane().add(create());
guiFrame.getContentPane().add(create());
guiFrame.pack(); // Resize frame to fit content
guiFrame.setLocationRelativeTo(null);
guiFrame.setVisible(true);
```

```
static int count = 0; // The counter which is incremented

static JLabel create()
{
    JLabel label = new JLabel("Hello World " + count++);
    label.setFont(new Font("Ariel", Font.BOLD, 80));
    return label;
}
```

Example: JLabel

Creating our own label which does something else…

Create a subclass of JLabel

```
import java.awt.Font;
import javax.swing.JLabel;

public class MyLabel extends JLabel
{
    public MyLabel()
    {
        setText("My label");
        setFont(new Font("Ariel", Font.BOLD, 30));
    }
}
```

- We can use the sub-class object as if it was the super-class object – i.e. we have setText() and setFont() methods!
  - Because inheritance models the 'is-a' relationship
- We configured the label in the constructor of the sub-class

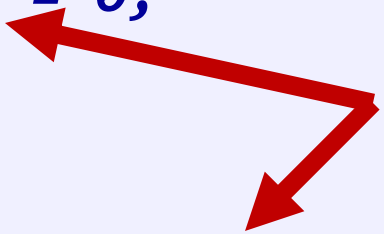
Using a superclass constructor

```
public class MyLabel extends JLabel
{
	public MyLabel()
	{
		super("My label");
		setFont(new Font("Ariel", Font.BOLD, 30));
	}

	public MyLabel( String str )
	{
		super( str );
		setFont(new Font("Ariel", Font.BOLD, 30));
	}
}
```

- We can pass parameters to the constructor for our superclass
- This will determine which constructor is used

Static and non-static counters

```
import java.awt.Font;
import javax.swing.JLabel;

public class MyLabel extends JLabel
{
    static int counter = 0;

    public MyLabel()
    {
       super("My label " + counter++ );
       setFont(new Font("Ariel", Font.BOLD, 30));
    }
}
```



- We can pass parameters to the constructor for our superclass
- This will determine which constructor is used

JLabel subclass – with an oval

- There are many other methods of JLabel that we can override
- For example, **`paintComponent()`**, the 'drawMyself' function:
  - We can change this function to change the appearance
  - It takes a parameter which is a reference to a graphics object, which we will use to do the drawing
- We can access super-class (base class) version of a method explicitly using '**`super.`**' if we wish (a lot like **`this.`**)

```
public void paintComponent(Graphics g)
{
    super.paintComponent(g);
    g.setColor(Color.RED);
    g.drawOval(0,0,getWidth()-1,getHeight()-1);
}
```

Demo using MyLabel

Creating a ColorLabel class

Note the US spelling

ColorLabel class : attributes

```
public class ColorLabel extends JLabel
{
    Color drawColor;
    Color borderColor;
    int borderSize;
```

The ColorLabel class will have three attributes:

- The colour to draw – or use null to not draw
- The width of the border
- The colour to draw the border – null if not drawing it

ColorLabel

- We will create two versions of the constructor, so we can:

1. Create a label which has a specified colour

```
new ColorLabel( 20, 20, Color.RED );
```

2. Create a label with both a main colour and a border of a different colour

```
new ColorLabel( 20, 20, Color.RED, 2, Color.GRAY );
```

- Parameters:
  - Width, Height: Size of label
  - Colour: colour to fill the label
  - Optional: Width and colour of the border

ColorLabel class : constructors

```
// First do the more complex constructor
public ColorLabel( int width, int height, Color color,
        int borderWidth, Color borderCol )
{
    // Member variable/attribute = parameter
    borderSize = borderWidth;
    drawColor = color;
    borderColor = borderCol;

    // Call some inherited methods from JLabel
    setMinimumSize( new Dimension(width, height) );
    setPreferredSize( new Dimension(width, height) );
}
```

Calling other constructors

- Simpler constructor just uses the complex one
- Keyword 'this' is used to call/delegate to other constructors

```
public ColorLabel( int width, int height, Color color )
{
     // Call the other constructor with some default values
     this( width, height, color, 0, null );
}

// A default constructor for testing
public ColorLabel()
{    // Again delegate work to the complex constructor
     this(200,200,Color.RED,5, Color.BLUE);
}
```

ColorLabel class : paintComponent()

```
    // This is where we do all of the work...
    protected void paintComponent(Graphics g)
    {
        //super.paintComponent(g); // Do NOT do normal drawing
        if ( borderColor != null )
        {
            g.setColor(borderColor);
            g.fillRect(0, 0, getWidth(), getHeight());
        }
        if ( drawColor != null )
        {
            g.setColor(drawColor);
            g.fillRect(borderSize, borderSize, getWidth()-
            borderSize*2, getHeight()-borderSize*2);
        }
    }
}
```

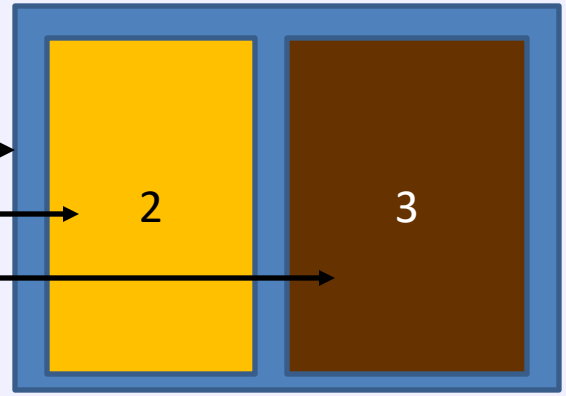
Demo using ColorLabel

Example Layout Managers

Demo using ColorLabel and layout managers

JPanel components – the concept

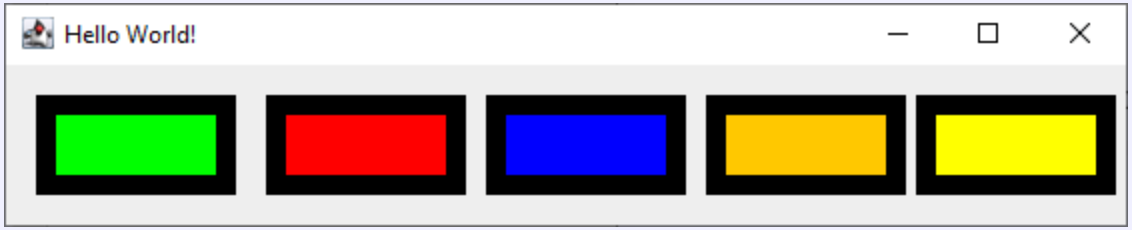
- Think of JPanel as a container for other components
- You can build up a user interface by putting JPanels inside JPanels

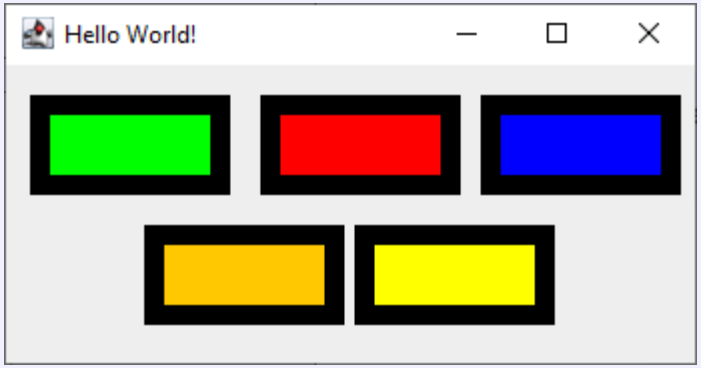



```
JPanel panel1 = new JPanel();
JPanel panel2 = new JPanel();
JPanel panel3 = new JPanel();

panel2.setLayout( new FlowLayout() );
panel1.add(panel2);
panel3.setLayout( new FlowLayout() );
panel1.add(panel3);

guiFrame.getContentPane().setLayout(new FlowLayout());
guiFrame.getContentPane().add(panel1);
```

Flow layout example

```
package pgp;

import java.awt.Color;
import java.awt.FlowLayout; // Remember ctrl-shift-o in Eclipse to generate these
import java.awt.Font;
import java.awt.GridLayout;

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

public class FlowLayoutExample
{
	public static void main(String[] args)
	{
		new FlowLayoutExample();
	}

	public FlowLayoutExample()
	{
		JFrame guiFrame = new JFrame();
		guiFrame.setTitle("Hello World!");

		JPanel panel1 = new JPanel();
		JPanel panel2 = new JPanel();
		JPanel panel3 = new JPanel();

		panel1.setLayout( new FlowLayout() );
		panel1.add(panel2);
		panel2.setLayout( new FlowLayout() );
		panel1.add(panel3);
		panel3.setLayout( new FlowLayout() );

		guiFrame.getContentPane().setLayout(new FlowLayout());
		guiFrame.getContentPane().add(panel1);
		guiFrame.getContentPane().add(
				new ColorLabel(100, 50, Color.ORANGE, 10, Color.BLACK));
		guiFrame.getContentPane().add(
				new ColorLabel(100, 50, Color.YELLOW, 10, Color.BLACK));

		panel1.add( new ColorLabel(100, 50, Color.BLUE, 10, Color.BLACK));
		panel2.add( new ColorLabel(100, 50, Color.GREEN, 10, Color.BLACK));
		panel3.add( new ColorLabel(100, 50, Color.RED, 10, Color.BLACK));

		guiFrame.pack(); // Resize frame to fit content
		guiFrame.setLocationRelativeTo(null);
		guiFrame.setVisible(true);
	}
}
```





Useful layout managers

- Common layouts:
  - FlowLayout()
  - BorderLayout()
  - GridLayout()
  - There are others as well
- When you add components you can give advice
  - E.g. placement in layout
- Remember to use US spellings, e.g. CENTER, COLOR etc

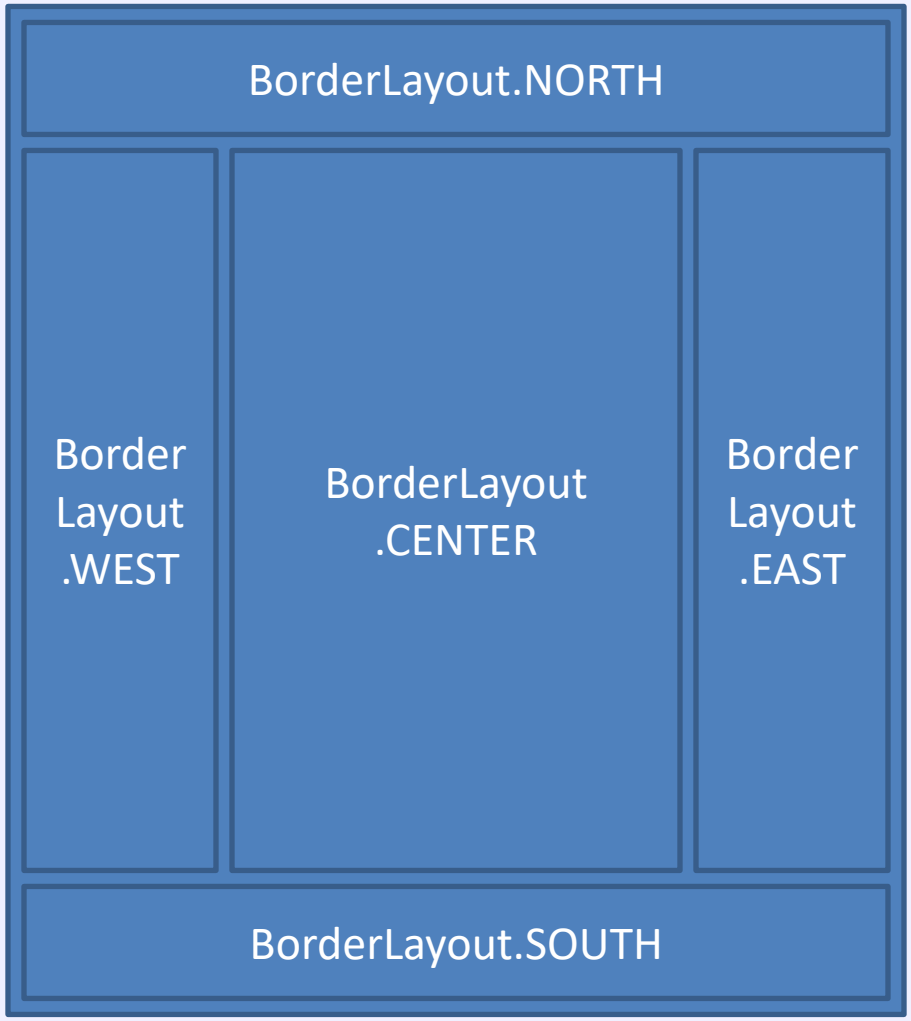


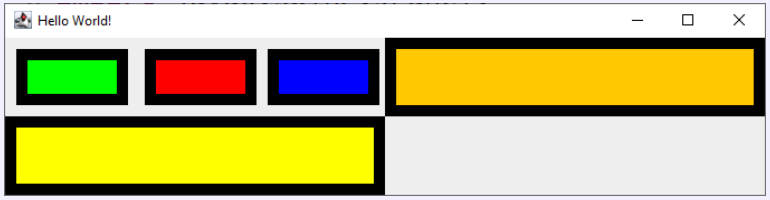
Diagram of how BorderLayout works

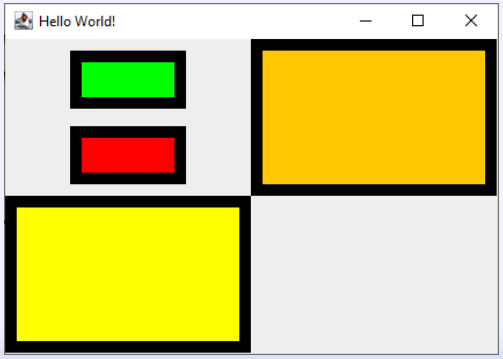
Component Sizes

- pack() resizes the frame to fit the contents
- When you resize the frame, sometimes components will also resize to fit it
- You can set minimum, maximum and preferred sizes for components
- Layout managers should not make them smaller than minimum or larger than maximum
- When container is trying to work out how big to make itself, it will use preferred sizes
- When multiple sizes are possible, preferred size will be ‘preferred’

Example code with grid layout

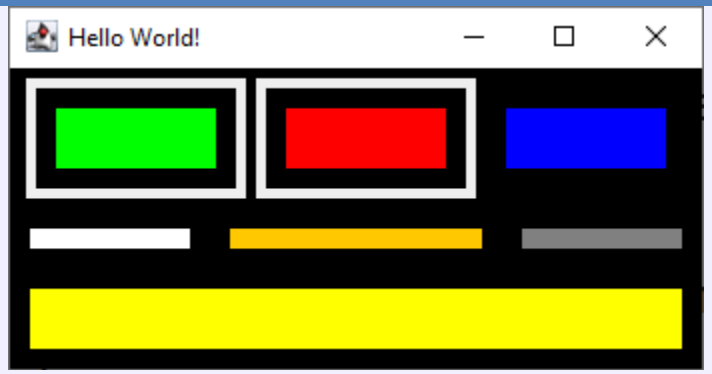
- **Change one line:**
  - **guiFrame.getContentPane().setLayout(new FlowLayout());**
- **To**
  - **guiFrame.getContentPane().setLayout(new GridLayout(2,2));**





Example code with border layout

- Try changing it to a BorderLayout
- But then every time we add something, we need to give a North/South/East/West clue




```
guiFrame.getContentPane().setLayout(new BorderLayout());

guiFrame.getContentPane().add(panel1,BorderLayout.NORTH);
guiFrame.getContentPane().add( new ColorLabel(100, 50,
Color.ORANGE, 10, Color.BLACK), BorderLayout.CENTER );
guiFrame.getContentPane().add( new ColorLabel(100, 50,
Color.YELLOW, 10, Color.BLACK), BorderLayout.SOUTH);
guiFrame.getContentPane().add( new ColorLabel(100, 50,
Color.WHITE, 10, Color.BLACK), BorderLayout.WEST);
guiFrame.getContentPane().add( new ColorLabel(100, 50,
Color.GRAY, 10, Color.BLACK), BorderLayout.EAST);
```

Strategy Pattern

- A ‘strategy pattern’ is a way to control the behaviour of one object using another
  - Allows objects’ *behaviour* to be changed at runtime
  - The strategies **should** be interchangable
    - i.e. the code to use it should be the same regardless of the layout manager used
    - Which layout manager you use will change that specific behaviour of the container
- Note: not actually the case for all layout managers – some use a *hint* object which is passed as a parameter to the add() method on the container
    - E.g. BorderLayout needs a hint about where to place the item: north, south, east, west or centre …
    - But understand the idea of interchangeability being important
- This is a known, common ‘design pattern’

Next Lecture

- Design Patterns
  - Layout managers are (sort-of) an example of the strategy pattern
    (swappable ‘components’/strategies which do something for the main class)
- Abstract classes and Interfaces
- JButtons and event handlers
  - An example of the Observer pattern

Lab 4

ColorLabel:
this lecture
Layout managers:
this lecture
Interfaces:
next lecture
Buttons:
next lecture

Hint:
Outside: BorderLayout
Grid in CENTER position
Button in SOUTH